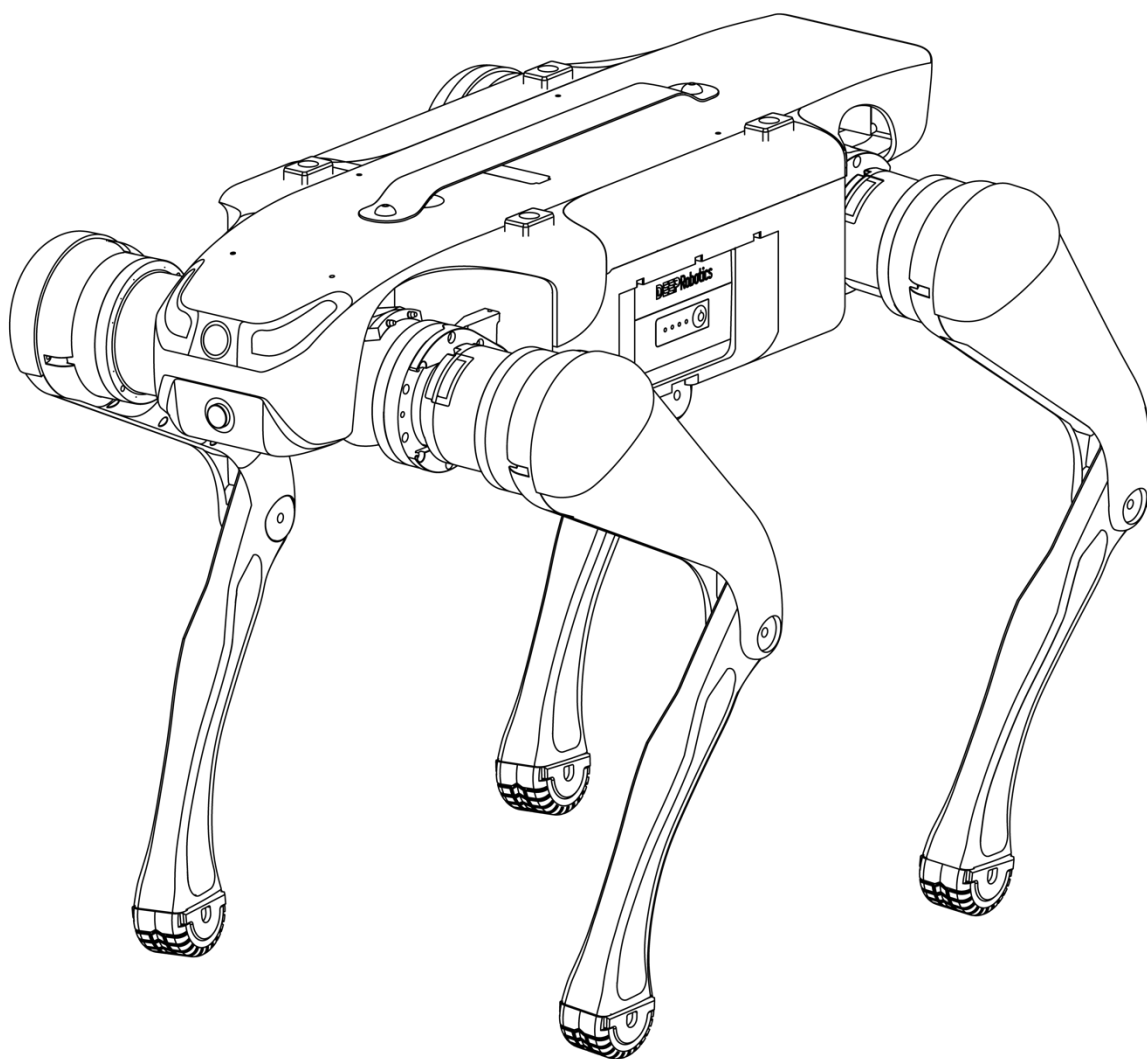


绝影 Lite3 运动开发手册(beta)

V2.2.0 2026.03.02



目录

1 运动系统	4
1.1 坐标系	4
1.1.1 身体坐标系	5
1.1.2 关节及足底坐标系	5
1.1.3 本体传感器坐标	6
1.2 规格参数	7
1.2.1 身体参数	7
1.2.2 腿部参数	8
1.2.3 关节硬件参数	8
1.2.4 关节性能参数	9
1.3 日志功能	9
1.4 系统保护	9
1.4.1 过温保护	9
1.4.2 摔倒保护	9
2 运动控制算法开发	10
2.1 强化学习运动控制算法训练与部署	10
2.2 传统运动控制 SDK	10
2.2.2 程序流程	11
2.2.3 开发步骤	12

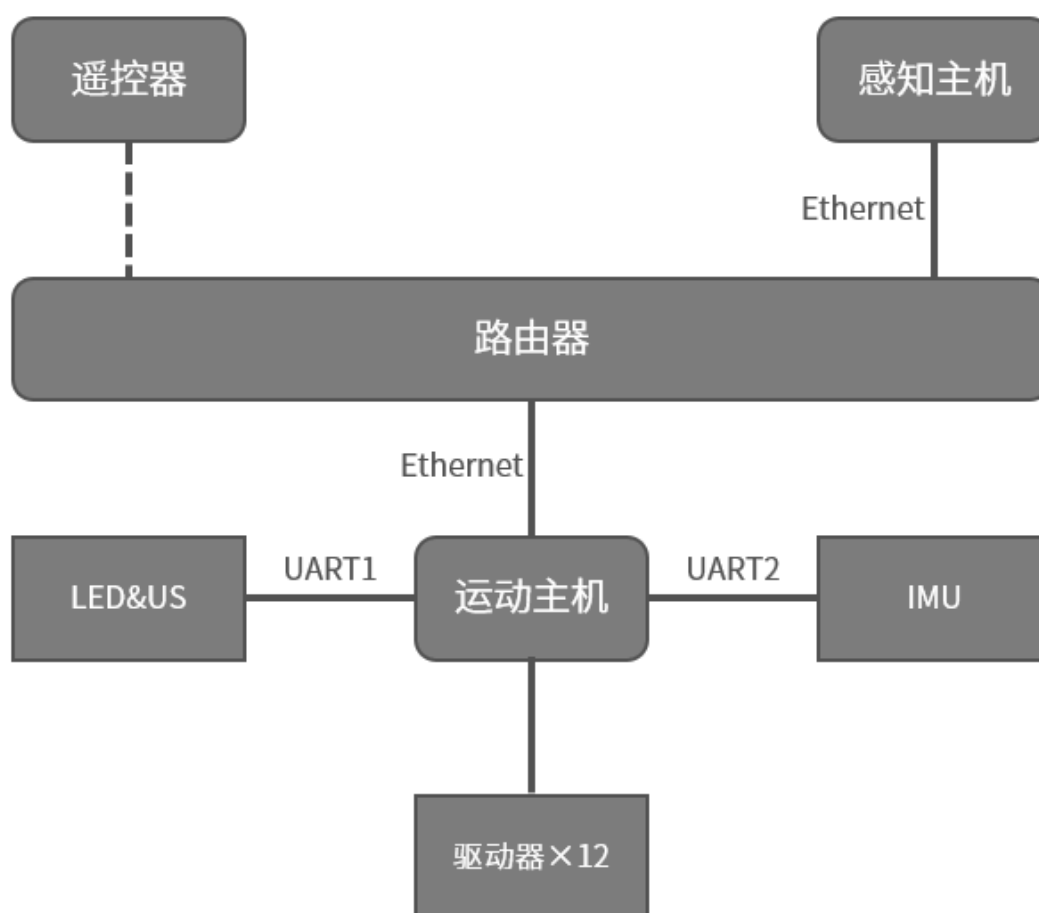
文档说明

本手册适用于需要通过绝影 Lite3 平台对运动控制算法进行探索、开发和验证，并且具备一定专业知识的用户。

手册版本	更新说明	发布日期
V1.0.0	对外发布	2023/4/18
V1.0.1	增加回调函数，修改保存数据到 robot_data	2023/5/23
V1.0.2	修改保存数据到 robot_data	2023/5/23
V1.0.3	修改身体参数	2023/6/16
V1.0.4	增加重启命令说明	2023/9/13
V2.0.0	修改开发步骤说明	2024/5/10
V2.0.1	修改关节运动范围	2024/7/8
V2.1.0	增加深度相机、激光雷达坐标信息	2025/11/14
V2.1.1	修改传感器与本体的连接方式说明	2025/12/30
V2.2.0	增加强化学习运动控制算法训练与部署	2026/3/2

1 运动系统

绝影 Lite3 运动主机选用 ARM 架构的 CPU 处理器 RK3588，并配有常用软件、依赖和库，以满足高动态运动控制和步态规划开发的需要。绝影 Lite3 由前左腿(FL)、前右腿(FR)、后左腿(HL)、后右腿(HR)和身体共同组成。每条腿由 3 个关节组成，从身体端开始依次是：髋侧摆关节(HipX)，髋前摆关节(HipY)和膝关节(Knee)。关节由大功率直流电机、精密减速机构和绝对式旋转编码器组成。

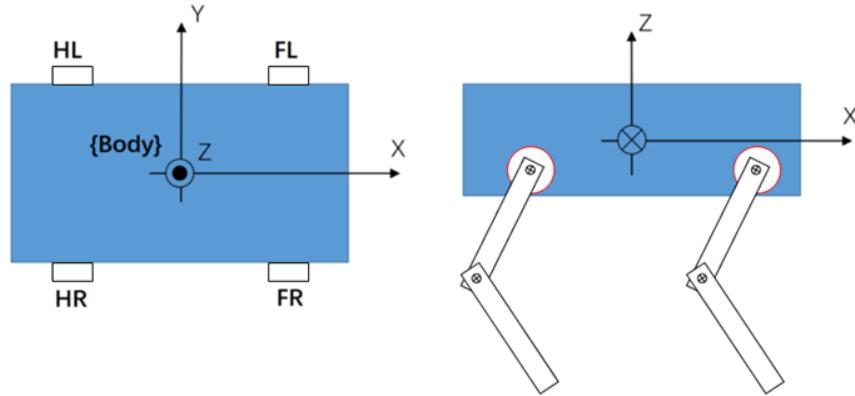


1.1 坐标系

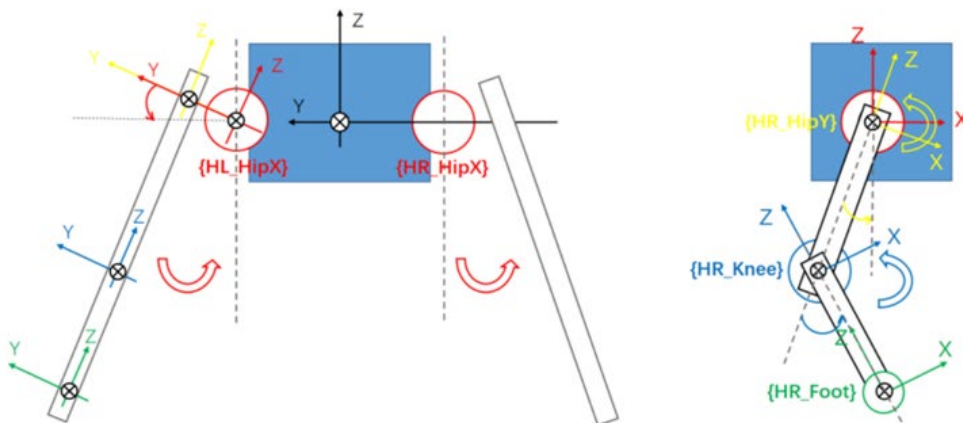
绝影 Lite3 采用三维坐标系来描述身体、传感器、关节和足底之间的位置关系。

1.1.1 身体坐标系

身体坐标系的原点位于机器狗身体的几何中心。



1.1.2 关节及足底坐标系



【注意】弧形箭头指示颜色相同的关节坐标系的旋转正方向。

坐标系	颜色	X(m)	Y(m)	Z(m)	备注
FL_HipX	红	0.1745	0.062	0	相对身体坐标系
FL_HipY	黄	0	0.0985	0	相对 FL_HipX 坐标系
FL_Knee	蓝	0	0	-0.20	相对 FL_HipY 坐标系
FL_Foot	绿	0	0	-0.21	相对 FL_Knee 坐标系
FR_HipX	红	0.1745	-0.062	0	相对身体坐标系
FR_HipY	黄	0	-0.0985	0	相对 FR_HipX 坐标系

FR_Knee	蓝	0	0	-0.20	相对 FR_HipY 坐标系
FR_Foot	绿	0	0	-0.21	相对 FR_Knee 坐标系
HL_HipX	红	-0.1745	0.062	0	相对身体坐标系
HL_HipY	黄	0	0.0985	0	相对 HL_HipX 坐标系
HL_Knee	蓝	0	0	-0.20	相对 HL_HipY 坐标系
HL_Foot	绿	0	0	-0.21	相对 HL_Knee 坐标系
HR_HipX	红	-0.1745	-0.062	0	相对身体坐标系
HR_HipY	黄	0	-0.0985	0	相对 HR_HipX 坐标系
HR_Knee	蓝	0	0	-0.20	相对 HR_HipY 坐标系
HR_Foot	绿	0	0	-0.21	相对 HR_Knee 坐标系

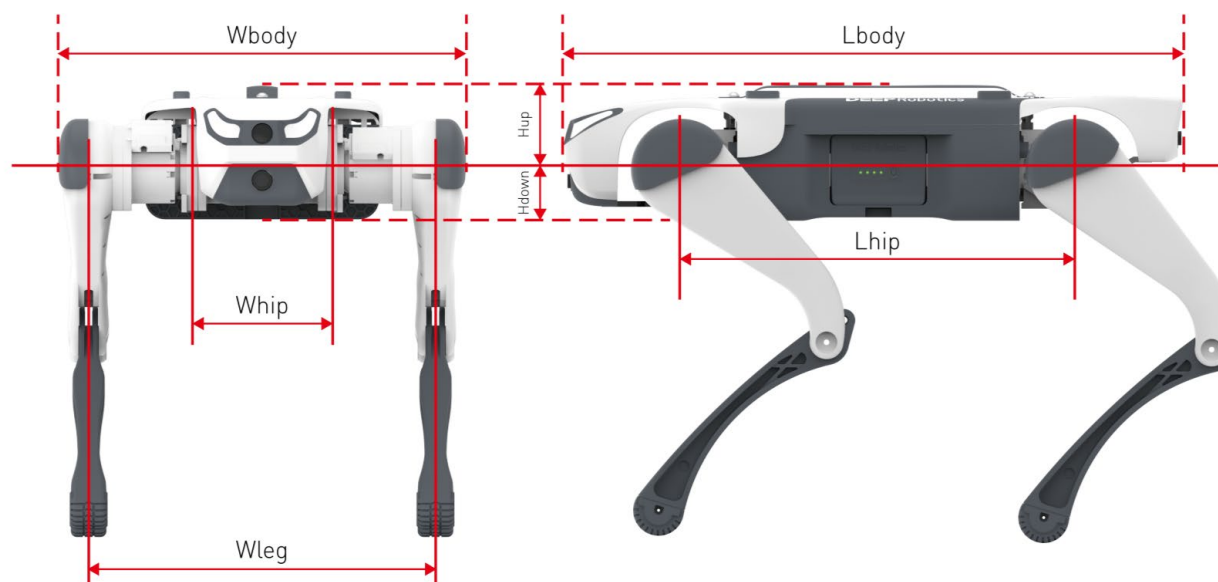
1.1.3 本体传感器坐标

本小节介绍绝影 Lite3 本体传感器坐标参数，其中 IMU 与运动主机直接连接；深度相机与感知主机直接连接；激光雷达通过交换机与感知主机连接：

传感器	坐标系数值（相对于身体坐标系）		
	X(mm)	Y(mm)	Z(mm)
IMU	5.00	0	0
深度相机（仅专业版和激光版包含）	-0.50	74.10	252.28
激光雷达（Mid 360，仅激光版包含）	0	124.98	186.78
激光雷达（RoboSense，仅激光版包含）	0	129.06	203.3

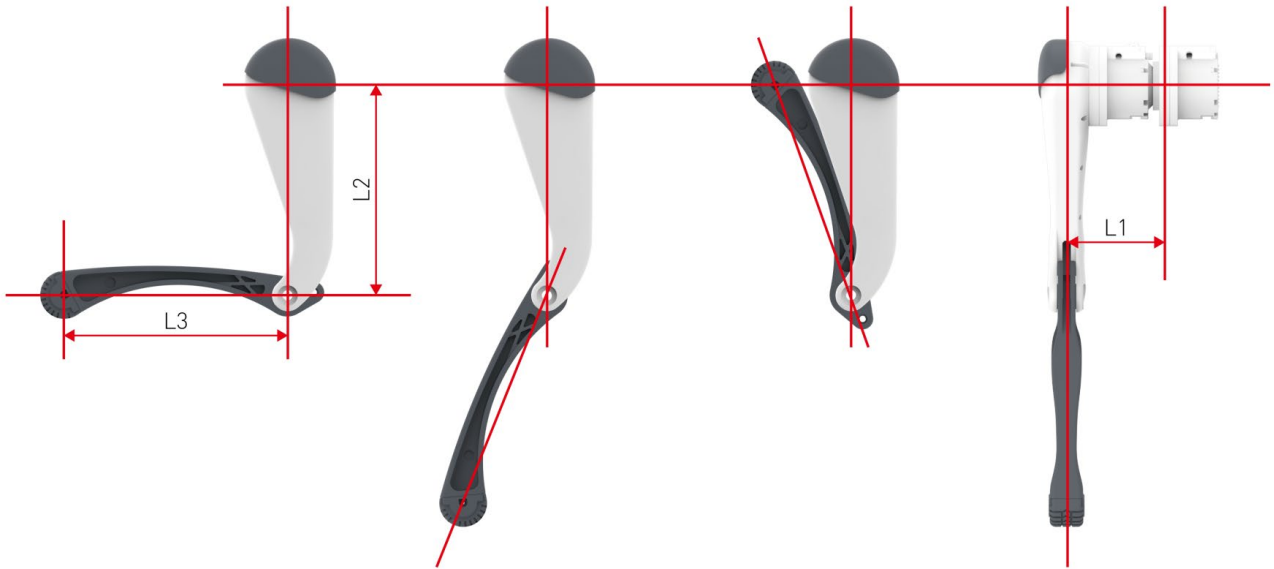
1.2 规格参数

1.2.1 身体参数



参数	数值	说明
重量 (mbody)	11kg	身体重量 (包括电池,侧摆)
长度 (Lbody)	548mm	身体总长度
宽度 (Wbody)	370mm	身体总宽度
身体顶部高度 (Hup)	74mm	身体顶部距离身体坐标系距离
身体底部高度 (Hdown)	44mm	身体底部距离身体坐标系距离
髌左右间距 (Whip)	124mm	左右髌关节中心距离
髌前后间距 (Lhip)	349mm	前后髌关节中心距离
腿平面左右间距 (Wleg)	328mm	腿平面左右距离

1.2.2 腿部参数



参数	数值	说明
单腿重量	1.12kg	腿部重量
腿平面与髋侧摆关节距离 L1	102mm	髋侧摆关节与腿平面距离
大腿长度 L2	200mm	髋前摆关节中心与膝关节中心距离
小腿长度 L3	210mm	膝关节中心与足底圆心距离
足底半径	20mm	足底缓冲件半径

1.2.3 关节硬件参数

关节	重量	惯性主力矩 Px	惯性主力矩 Py	惯性主力矩 Pz
髋侧摆(HipX)	0.435kg	220kg·mm ²	220kg·mm ²	254kg·mm ²
髋前摆(HipY)	0.501kg	267kg·mm ²	316kg·mm ²	369kg·mm ²
膝关节(Knee)+大腿	0.96kg	945kg·mm ²	4440kg·mm ²	4692kg·mm ²
小腿	0.16kg	32kg·mm ²	1294kg·mm ²	1310kg·mm ²

1.2.4 关节性能参数

关节	运动范围	额定转矩	峰值转矩	峰值转速
髋侧摆(HipX)	-24°~24°	6.48N·m	24N·m	26.16rad/s
髋前摆(HipY)	-200°~20°	6.48N·m	24N·m	26.1rad/s
膝关节(Knee)	34.5°~156°	9.72N·m	36N·m	17.1rad/s

1.3 日志功能

系统在运动过程中会产生三种日志文件，都会保存在机器人内部主机硬盘的 `/jy_exe/data` 目录下。

- 事件日志(.log)：机器人运行过程每个事件都会被记录此文件中。
- 运算日志(.csv)及故障快照(.snapshot.csv)压缩包：机器人在运行中如果出现故障，请在 APP 上使用“保存数据”功能，机器人将保存故障数据并停止运动程序。日志文件默认的记录频率是 1000Hz，每一次记录的数据都可以用 Microsoft Excel 直接查看。

1.4 系统保护

1.4.1 过温保护

运动系统自带温度感知，一旦机器人长时间运行导致电机或者驱动器过热，将自动进入过温保护状态，机器人将自动停止运动，原地趴下。

1.4.2 摔倒保护

当 IMU 模块检测到姿态角度改变过大，机器狗无法避免摔倒时，会立刻使各关节处于阻尼状态。请在确认周围无障碍物后再使其翻身爬起。若无法爬起，按下软急停按钮，然后在 APP “设置” 页面中保存数据或直接关机。

2 运动控制算法开发

2.1 强化学习运动控制算法训练与部署

强化学习策略的[训练](#)与[部署](#)代码已上传至 [GitHub](#) 仓库，相关训练与部署教学视频已同步发布至 [Bilibili](#) 平台。

2.2 传统运动控制 SDK

绝影 Lite3 运动控制 SDK：获取每个关节的数据，并下发数据，控制机器人运动。

其中 SDK 结构如下所示：

```
.
├─ CMakeLists.txt
├─ include
│   ├── common
│   ├── dr_timer.h
│   ├── motionexample.h
│   ├── receiver.h
│   ├── robot_types.h
│   └─ sender.h
├─ lib
│   ├── eigen3
│   ├── libdeeprobotics_legged_sdk_aarch64.so
│   └─ libdeeprobotics_legged_sdk_x86_64.so
├─ main.cpp
└─ src
    └─ motionexample.cpp
```

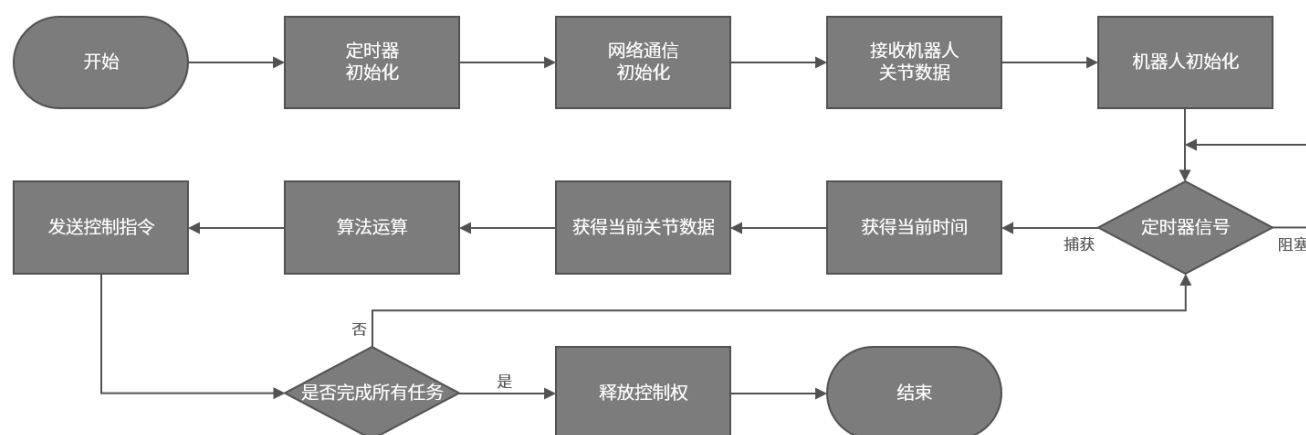
上述程序包含：

1. CMakeLists 编译工具，在 ARM 和 x86 两种不同的架构中，需要稍作修改；
2. include 文件夹下是关于接收，发送，运动算法，定时器和通用工具；
3. lib 文件夹下是运动算法所需要的库，x86，ARM 的动态库；
4. main.cpp 是主要的运行代码；

5. src 下是运动算法的主要 demo。

进行二次开发时，通过修改开发包中 `main.cpp` 中的代码，控制关节，从而实现新的机器人动作。

2.2.2 程序流程



1. 定时器初始化：创建定时器 `DRTimer`，并将定时器周期设置为 1（算法周期）；
2. 网络通信初始化：创建 `Sender` 和 `Receiver`，用于发送数据与接收数据；
3. 机器人初始化：关节回零（获得控制权）；
4. 获取当前时间戳（用于算法计算）；
5. 获取每个关节数据 `RobotData`（至此，准备工作完成）；
6. 进入主循环：
 - 等待定时器触发；
 - 获取当前时间；
 - 获取关节状态及数据；
 - 算法运算 `MotionExample`（含示例代码）；
 - 发送关节控制指令；

（完成上述算法期望动作后，释放控制权）；

7. 结束。

2.2.3 开发步骤

开发步骤请参考 [Lite3_MotionSDK 代码仓库](#)。